Lecture 2 - DSA 4.03.2026

- **RECORD (STRUCT):**
  - → static data structure
  - ex:

Person
name: String
dob: String
addr: String
etc...

- **ARRAY**
  - → data structure (static)
  - → holds fixed nr. of elements of the same type
  - → elements occupy a contiguous memory block.
  - → used as a basis for other data structures.
  - → when creating an array, SPECIFY:
    1) the TYPE of elements.
    2) the MAX nr. of elements (CAPACITY)
  - => memory occupied by array:
    CAPACITY * sizeof(elem)
  - → address of the first element.

ARRAY of boolean values

↳ addresses of the elements are displayed in b16, b10.

ADVANTAGES OF ARRAYS:

- (+) elem. can be accessed in $\Theta(1)$ (constant time)

address of i^{th} elem = address of array $+ i *$ sizeof(elem)

↳ indexing from 0

↳ indexing from 1: " −1 " — (i−1) " −1 " — ...

DISADVANTAGES:

- we know from the start the number of elements.

DYNAMIC ARRAYS (vectors)

- their size can grow / shrink
- contiguous memory locations
- compute address of every element in $\Theta(1)$ time.

DynamicArray:
cap: Integer
nrElem: Integer
elems: TElem[]

- cap → max capacity (number of slots allocated for the array)
- nrElem → current capacity (actual nr. of elements stored)
- elems → actual array with cap slots for TElem allocated.

?

- have positions (no iterators needed)
- they are data structures (ADT) :)

- FULL ARRAY => mrElem = cap.
- More elements needed? => cap * 2. => resized.

RESIZE OPERATION:

→ resize before add

old: |||||||| (fixed)

copy ↓

new: |||||||| (larger)

→ resize after delete (be it's "too empty") => shrink.
(to avoid occupying unused memory).

Why is Dynamic Array a DS?

- it describes how data is stored in computer
- it can be used as representation to implement ≠ ADT's.

Why not really an ADT?

- it has one single possible implementation → ?!
- but we treat it as an ADT and discuss its interface.

Formal definition for Dynamic Array (ADT):

- Domain: $DA = \{da \mid da = (cap, mrElem, e_1 e_2 \dots e_{mrElem}), cap, mrElem \in \mathbb{N}, mrElem \leq cap, e_i \text{ is of type TElem}\}$.
- Interface:

Operations for DA:

1) • init(da, cp) ← CONSTRUCTOR

↳ creates new, empty Dynamic Array with initial capacity (cp).

- pre: $cp \in \mathbb{N}^*$
- post: $da \in DA$
  $da.cap = cp$
  $da.mrElem = 0$
- throws: an exception if ~~cp = 0 or~~ $cp \leq 0$.

2) • destroy(da) ← DESTRUCTOR

↳ destructor

- pre: $da \in DA$
- post: da was destroyed (memory occupied was freed).

3) • size(da) $\Theta(1)$

↳ nr. of elements of the DA.

• pre: $da \in DA$

• post: size ← nr. of elements ...

4) • getElement(da, i) $\Theta(1)$

↳ returns element from a position from the DA.

• pre: $da \in DA$, $1 \leq i \leq da.nrElem$

• post: getElement ← e, $e \in TElem$

$e = da.e_i$ (element from position i)

• throws: exception if i not valid pos.

5) • setElement(da, i, e) $\Theta(1)$

↳ changes the element from a position to another value

• pre:

• post:

• throws: " — "

look at the sequence of calls → compute average nr. of instructions ... (not AC); in time, divided by sth; those costs that don't do resizing.

6) • addToEnd(da, e) $\Theta(1)$ amortized

↳ adds to the end of DA; if full, increases its capacity

• pre: $da \in DA$, $e \in TElem$

• post: $da' \in DA$, $da'.nrElem = da.nrElem + 1$

$da'.e_{da'.nrElem} = e$

$da.cap = da.nrElem$

⇓ increase size

$da'.cap \leftarrow da.cap * 2$

ADD

PSEUDOCODE IMPLEMENTATION:

full ⇔ capacity = nr. elems

```
subalgorithm addToEnd(da, e):
    if da.cap = da.nrElems then
        da.cap ← da.cap * 2
        newArray ← @ is a new array with da.cap elements
        for i ← 1, da.nrElems execute
            newArray[i] ← da.elems[i]
        @deallocate da.elems
        da.elems ← newArray
    da.nrElems ← da.nrElems + 1
    da.elems[da.NrElems] ← e
```

no variable holds the address of the old array

7) • addToPosition(da, i, e) O(n)

↳ adds elem to given position.

• pre: $da \in DA$, ...

• post: $da' \in DA$

$da'.nrElem = da.nrElem + 1$

$da'.e_j = da.e_{j-1}$, $(\forall) j$ ...

...

• throws: exception if i is not a valid pos.

! $da.nrElem + 1$ is a valid position when adding new elem.

⇒ BC, AC, WC

↓

resize

- ! valid position: nr. from 1 → nrElem
- valid position when adding: nr from 1 → nrElem + 1.
- * 2 too much when resizing?

↳ we need to multiply always, at least with $ct \geq 1$. (in C++, Java) ... does not change, then?

↳ it is better to RESIZE ONCE (to capacity 100, for instance) than to go through a lot of ≠ capacities to get to 100.

<u>what happens in memory:</u>

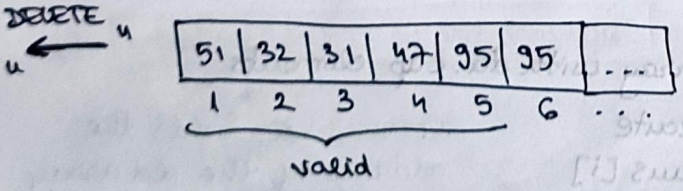
dynamic array → variable stores <u>address of allocated memory</u>.

→ elements stored there → when capacity exceeded → new larger block is allocated, elements copied, old block freed → pointer updated to new address.

8) • deleteFromPosition(da, i) O(n)

↳ ....

DELETE



Complexity: size: $\Theta(1)$
getElement: $\Theta(1)$
setElement: $\Theta(1)$
iterator: $\Theta(1)$
addToPosition: $O(n)$
deleteFromEnd: $\Theta(1)$
deleteFromPosition: $O(n)$
deleteGivenElement: $\Theta(n)$
addToEnd: $\Theta(1)$ amortized → better explanation:

BC for searching = WC for removing
G (WC: searched elem. is on last position).

9) • iterator(da, it)
↳ returns an iterator for the DA.
• pre: da ∈ DA
• post: (it) ∈ I is an iterator over da
↳ current elem from (it) = first element from da ?